Запуск симулятора Gazebo VRX в Docker с Ubuntu

Проект: Симуляционная среда для автономного надводного судна (USV)
Цель: Запуск Gazebo Harmonic с VRX-плагинами и моделью катера WAM-V в Docker-контейнере
Статус: ✅ Выполнено
Версия документа: 2.0 (обновлено 02.04.2026)

---

1. Попытка запуска на RED OS — причины перехода

Что такое RED OS и почему использовалась

RED OS — отечественный дистрибутив Linux на базе RPM (разработка компании «Ред Софт»), широко применяемый в российских университетах и государственных учреждениях в рамках политики импортозамещения ПО. Именно по этой причине целевой рабочей станцией являлся ПК с RED OS.

Причины неуспешной попытки

При попытке развернуть стек ROS2 + Gazebo + Docker на RED OS возник ряд системных ограничений:

- **Отсутствие поддержки rootless Docker.** Rootless-режим требует наличия маппингов /etc/subuid и /etc/subgid для пользователя (минимум 65 536 подчинённых UID/GID). На корпоративном/университетском RED OS эти маппинги отсутствовали и не могли быть добавлены без прав администратора.
- **Строгие политики безопасности SELinux.** RED OS по умолчанию использует SELinux в enforcing-режиме, что

блокировало монтирование `/tmp/.X11-unix` и работу GPU-ускорения в контейнерах.
- **Отсутствие пакетов ROS2 Jazzy в репозиториях.** Официальные репозитории ROS2 ориентированы на Debian/Ubuntu; для RPM-дистрибутивов (включая RED OS) сборки отсутствуют, а ручная компиляция из исходников в ограниченной среде оказалась нежизнеспособной.
- **Несовместимость версии ядра с NVIDIA Container Toolkit.** NVIDIA Container Toolkit, необходимый для передачи GPU в контейнер, требует специфических версий ядра и libc, которые в тестируемой конфигурации RED OS не поддерживались.
- **Уровень доступа.** Пользователь не имел прав `sudo`, что исключало установку системных зависимостей и изменение конфигурации Docker-демона.

Решение — переход на Ubuntu

Был использован другой рабочий ПК в том же помещении с предустановленной Ubuntu 24.04.4 LTS. Ubuntu является официально поддерживаемой платформой для ROS2 Jazzy, NVIDIA Container Toolkit и Gazebo Harmonic, что позволило сразу перейти к развёртыванию симулятора без борьбы с системными ограничениями.

---

2. Характеристики рабочего стенда

Аппаратное обеспечение и программная среда

Параметр	Значение
Тип машины	Университетский стационарный ПК
Операционная система	Ubuntu 24.04.4 LTS
GPU	NVIDIA GeForce RTX 4070
Доступ к sudo	Имеется
Группа пользователя	`docker` (добавлен администратором)
Режим Docker	Rootful (системный демон)
`unprivileged_userns_clone`	✅ Включён (1)
subuid/subgid маппинги	❌ Отсутствуют
ROS2 дистрибутив	Jazzy Jalisco
Gazebo	Harmonic (совместимая с ROS2 Jazzy)
Контейнеризация	Docker + Docker Compose

Особенности конфигурации

Все операции с симулятором выполняются через `docker` (благодаря членству в группе docker). Rootful-режим означает, что системный Docker-демон работает от `root`, при этом пользователь управляет контейнерами без `sudo`. Отсутствие `subuid/subgid` маппингов означает, что rootless Docker использовать невозможно — это нормально для текущей схемы работы.

Передача дисплея осуществляется через X11 forwarding (/tmp/.X11-unix). GPU в контейнере пробрасывается через NVIDIA Container Toolkit с `driver: nvidia` в `docker-compose.yml`.

---

3. Структура репозитория и ROS2-пакетов

После сборки воркспейса внутри контейнера в `/ros2_ws/install/` присутствуют следующие ключевые пакеты:

```
/ros2_ws/install/
├── wamv_description/      # Описание модели катера WAM-V (URDF, мес
│   └── share/wamv_description/
│       └── models/
│           ├── WAM-V-Base/mesh/WAM-V-Base.dae
│           ├── engine/mesh/engine.dae
│           └── propeller/mesh/propeller.dae
├── wamv_gazebo/           # Gazebo-плагины и конфиги катера
│   └── share/wamv_gazebo/
│       └── models/
│           └── gps/mesh/gps.dae
├── vrx_gz/                # Основной VRX пакет (миры, задания, модели)
│   └── share/vrx_gz/
│       ├── models/
│       └── worlds/
└── vrx_gazebo/            # Дополнительные ресурсы VRX
    └── share/vrx_gazebo/
        └── models/
```

Пакеты `wamv_description` и `wamv_gazebo` берутся из репозитория `osrf/vrx`. Сборка выполняется через `colcon build` — стандартный инструмент сборки ROS2.

---

4. Актуальная инструкция по сборке и запуску

4.1 Требования к хост-системе

- ОС: Ubuntu 24.04 LTS (рекомендуется, проверено)
- GPU: NVIDIA (тестировалось на RTX 4070)
- Docker: версия 20.10+ (rootful-режим)
- Docker Compose: v2+

Проверить готовность:

```
docker run --gpus all nvidia/cuda:12.0-base nvidia-smi   # Должна показати
xhost +local:docker                                      # Разрешить X11 для Docker
```

4.2 Структура проекта

> ⚠ **Актуальное название папки проекта: `usv_project`**

```
usv_project/
├── debs/                # .deb-пакеты, которые отсутствуют в apt
│   ├── libgz-math7_7.5.1-1~noble_amd64.deb
│   ├── python3-gz-math7_7.5.1-1~noble_amd64.deb
│   └── python3-sdformat14_14.9.0-1~noble_amd64.deb
├── Dockerfile
└── docker-compose.yml
```

Зачем папка `debs/`? Пакеты `libgz-math7`, `python3-gz-math7` и `python3-sdformat14` требуются для сборки VRX, но система безопасности САФУ не позволяет их установить напрямую. По этой причине они загружаются вручную и копируются в образ через Dockerfile.

4.3 Dockerfile (актуальный)

```
FROM osrf/ros:jazzy-desktop-full
ENV DEBIAN_FRONTEND=noninteractive

COPY debs/ /tmp/debs/

RUN apt-get update && apt-get install -y --no-install-recommends \
    python3-colcon-common-extensions \
    python3-pybind11 \
    git build-essential tzdata \
    ros-jazzy-xacro \
    ros-jazzy-ros-gz-interfaces \
    && dpkg -i /tmp/debs/libgz-math7_7.5.1-1~noble_amd64.deb \
```

```
    && dpkg -i /tmp/debs/python3-gz-math7_7.5.1-1~noble_amd64.deb \
    && dpkg -i /tmp/debs/python3-sdformat14_14.9.0-1~noble_amd64.deb \
    && apt-get install -f -y \
    && rm -rf /var/lib/apt/lists/* /tmp/debs

RUN mkdir -p /ros2_ws/src
WORKDIR /ros2_ws

RUN git clone --depth 1 -b jazzy https://github.com/osrf/vrx.git src/vrx

RUN /bin/bash -c "source /opt/ros/jazzy/setup.bash && \
    rosdep init || true && \
    rosdep update && \
    apt-get update && \
    rosdep install --from-paths src --ignore-src -r -y \
      --skip-keys='ament_cmake_pycodestyle' && \
    rm -rf /var/lib/apt/lists/*"

RUN /bin/bash -c "source /opt/ros/jazzy/setup.bash && \
    colcon build \
      --symlink-install \
      --parallel-workers $(nproc) \
      --event-handlers console_direct+"

RUN echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc && \
    echo 'source /ros2_ws/install/setup.bash' >> ~/.bashrc

ENV ROS_DOMAIN_ID=0
ENV QT_X11_NO_MITSHM=1
ENV QT_LOGGING_RULES="*.debug=false;qt.*.debug=false"

CMD ["bash"]
```

4.4 docker-compose.yml (актуальный)

```yaml
services:
  usv-sim:
    build: .
    container_name: usv_simulator
    stdin_open: true
    tty: true
    restart: unless-stopped
    environment:
      - DISPLAY=${DISPLAY}
      - QT_X11_NO_MITSHM=1
      - NVIDIA_DRIVER_CAPABILITIES=compute,utility,graphics,display
      - LIBGL_ALWAYS_INDIRECT=0
      - GZ_SIM_RESOURCE_PATH=/ros2_ws/install/wamv_description/share:/ros
    volumes:
      - /tmp/.X11-unix:/tmp/.X11-unix:rw
      - /etc/localtime:/etc/localtime:ro
    network_mode: host
    ipc: host
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
```

> ⚠ **Важно:** `GZ_SIM_RESOURCE_PATH` указана и в Dockerfile, и в `docker-compose.yml`. Значение в `docker-compose.yml` переопределяет ENV из образа — менять пути можно **без пересборки** образа, достаточно `docker compose down && docker compose up -d`.

> ⚠ **Важно:** `docker compose restart` **не применяет** изменения переменных `environment:`. Всегда используйте `docker compose down && docker compose up -d`.

> ⚠ **Важно:** Строки `DISPLAY=${DISPLAY}` и том `/tmp/.X11-unix` **обязательны**. Их удаление приводит к ошибке `Failed to create OpenGL context` при запуске Gazebo.

4.5 Сборка и запуск

Первый запуск (сборка образа — занимает 10–30 минут):

```
# 1. Разрешить X11 соединения для Docker (выполнять перед каждой сес
xhost +local:docker

# 2. Перейти в директорию проекта
cd usv_project/

# 3. Собрать образ (только при первом запуске или после изменений Doc
docker compose build

# 4. Запустить контейнер
docker compose up -d

# 5. Войти в контейнер
docker exec -it usv_simulator bash

# 6. Запустить симулятор
ros2 launch vrx_gz competition.launch.py
```

Последующие запуски (без пересборки):

```
xhost +local:docker
cd usv_project/
docker compose up -d
docker exec -it usv_simulator bash
ros2 launch vrx_gz competition.launch.py
```

4.6 Изменение GZ_SIM_RESOURCE_PATH без пересборки

Если пути к ресурсам изменились (например, после добавления нового пакета):

```
# Отредактировать docker-compose.yml (поле environment: GZ_SIM_RESO
nano docker-compose.yml

# Применить без пересборки образа
docker compose down
docker compose up -d
```

4.7 Диагностика при проблемах

Gazebo не запускается — ошибка OpenGL при старте

Симптом: при выполнении `ros2 launch vrx_gz competition.launch.py` процесс сразу завершается с сообщением `Failed to create OpenGL context` или `Aborted (core dumped)`.

```
# Шаг 1: Выполнить на ХОСТЕ (не в контейнере) — разрешить X11
xhost +local:docker

# Шаг 2: Перезапустить контейнер (restart недостаточно!)
docker compose down && docker compose up -d

# Шаг 3: Войти и проверить GPU
docker exec -it usv_simulator bash
nvidia-smi

# Шаг 4: Проверить переменную DISPLAY внутри контейнера
echo $DISPLAY    # Должна выводить что-то вроде :1
```

WAM-V не отображается (меши отсутствуют)

```
# 1. Убедиться, что меши физически существуют
find /ros2_ws/install -name "*.dae" | grep wamv

# 2. Проверить текущий ресурс-путь
echo $GZ_SIM_RESOURCE_PATH

# 3. Проверить, что Gazebo видит пакеты
ros2 pkg prefix wamv_description
ros2 pkg prefix vrx_gz

# 4. Проверить GPU внутри контейнера
nvidia-smi
```

---

5. Проверка успешности запуска

После выполнения `ros2 launch vrx_gz competition.launch.py` должны быть выполнены все три условия:

1. **Окно Gazebo открывается** — видны море, небо, освещение
2. **Катер WAM-V виден** — модель с корпусом, двигателями и пропеллерами отображается в сцене
3. **Топики ROS2 активны** — `ros2 topic list` показывает топики `/wamv/...`

```
# Проверить доступные топики катера
ros2 topic list | grep wamv
```

---

6. Визуализация в RViz2

6.1 Запуск RViz2

RViz2 запускается **внутри контейнера** в отдельном терминале (симулятор должен быть уже запущен):

```
docker exec -it usv_simulator bash
rviz2
```

6.2 Обязательные настройки RViz2

После открытия RViz2 необходимо настроить три параметра — без них модель не отобразится:

Параметр	Где найти	Правильное значение
Fixed Frame	Панель «Global Options» (левая колонка)	`wamv/wamv/base_link`
Description Topic	Дисплей «RobotModel» → поле Topic	/wamv/robot_description
TF Prefix	Дисплей «RobotModel» → поле TF Prefix	`wamv`

> ⚠ **Почему двойной префикс в Fixed Frame?**
> Топики публикуются с пространством имён `wamv/wamv/...` — штатное поведение VRX. Если указать просто `base_link` или `wamv/base_link`, все TF-фреймы дадут ошибку `No transform` и модель будет белой/невидимой.

> ⚠ **Почему нестандартный Description Topic?**
> URDF катера публикуется в `/wamv/robot_description`, а не в дефолтный `/robot_description`. Без явного указания RViz2 не найдёт описание модели.

6.3 Признаки корректной работы RViz2

- Модель WAM-V отображается полностью (корпус, двигатели, пропеллеры)
- В панели «Displays» у RobotModel нет красных ошибок
- TF-фреймы видны в сцене (если включён дисплей TF)

References

1. Rootless mode - Docker Docs — Run the Docker daemon as a non-root user (Rootless mode)
2. Installing Gazebo with ROS
3. Запустите демон Docker как пользователь без прав root
4. osrf/vrx: Virtual RobotX (VRX) resources. — GitHub
5. Quick start
6. ros_gz_sim: Jazzy 1.0.21 documentation
7. ros_gz_sim - ROS Package Overview
8. Build without --merge-install flag #746 — osrf/vrx — GitHub
9. Can I change the environment setting without rebuilding the docker image?
10. Elegant way to reload env_file within docker-compose?
11. Discovery fails if daemon is started as root · Issue #598 · ros2/ros2cli